POO COM C++

Pedro Henrique Ribeiro Dias

Inatel

INTRODUÇÃO

A programação orientada a objetos (POO) organiza código em torno de objetos reais, facilitando a modelagem de sistemas complexos. Podemos especificar as características e comportamentos desses objetos, além de estabelecer relações entre eles.

POO COM C++

1. Permite melhor organização, reutilização de código e manutenção facilitada, promovendo sistemas altamente escaláveis e flexíveis.

2. Criamos classes como moldes para objetos que possuem atributos e métodos, ou seja, características e comportamentos

PILARES DA POO

1. Abstração
2. Herança
3. Encapsulamento
4. Polimorfismo

ABSTRAÇÃO

Abstraímos Entidades do mundo real para nosso código, dando a elas caraterísticas e comportamentos que representem a realidade.

·Exemplo:

- Identidade : Aluno.
- Propriedades: Nome, Idade, Matrícula.
- Métodos: Questionar, Estudar , Surtar.

HERANÇA

Uma classe pode "herdar" características de outra. Isso evita repetir código, portanto atributos da classe “mãe” são passados para a classe filha.

·Exemplo

- Uma classe Professor pode herdar de Ser Humano. Com isso atributos (nome, idade) e métodos (Falar, Comer) da classe Ser Humano seriam passados para Professor.

ENCAPSULAMENTO

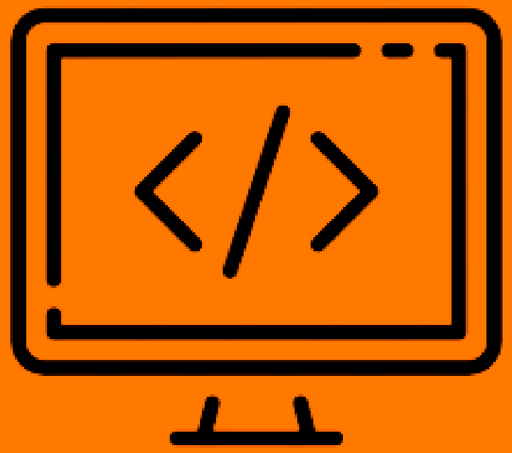
Ao definirmos os atributos de um objeto, devemos garantir que eles sejam "trancados", ou seja, o objeto pode ter suas propriedades consultadas, mas não modificadas por outros objetos.

POLIMORFISMO

É as características de dois ou mais objetos derivados de uma mesma classe possuírem respostas diferentes para cada método.

Exemplo:

- Classes Animal possui um método para emitir som. Suas classes filhas, como cachorro e gato, teriam esse método diferente, pois emitem diferentes sons.



KEEP CALM

AND

PROGRAME ORIENTADO A OBJETOS

EXERCÍCIO 1

Cenário: Em uma competição de tecnologia avançada, cada robô possui um modelo, uma versão de software, uma potência de laser e um nível de integridade do chassi.

Tarefa:

- Crie uma classe Robô.
- Adicione os atributos: modelo (string), versao (int), potenciaLaser (float) e integridade (int).
- Crie um método chamado disparar(): Ele deve receber como parâmetro um objeto da classe Robô (o alvo). O método deve exibir uma mensagem confirmando o disparo e subtrair o valor da potenciaLaser da integridade do robô atingido.
- Na main: Instancie 2 objetos da classe Robô.
  - Atribua valores personalizados aos atributos de cada um.
  - Execute o método disparar(), definindo qual robô será o atacante e qual será o alvo.
  - Ao final, exiba o status atualizado de ambos os robôs para ver como ficou a integridade após o confronto.

EXERCÍCIO 2

Cenário: No jogo Persona, cada personagem, seja o protagonista ou um de seus aliados, tem características pessoais que devem ser protegidas e alteradas de forma controlada.

Tarefa:

1. Crie uma classe base chamada Pessoa.
   - Adicione os atributos nome (string) e idade(int) como privados.
2. Crie uma classe Protagonista e outra Personagem
   - Ambas herdeiras de Pessoa. Protagonista deve ter um atributo de nível(int) e Personagem deve ter um atributo de rank(int)
   - (obs: o rank dos Personagens/social link varia de 0 a 10).
   - Ambos devem ser privados.
3. Na main:
   - crie um objeto da classe Protagonista e atribua um nome e nível.
   - Faça o mesmo para um objeto da classe Personagem.
   - Imprima os valores e demonstre como os atributos estão sendo acessados .

EXERCÍCIO 3

Cenário: No Inatel, durante eventos como a FETIN, diferentes membros da instituição desempenham papéis distintos. Todos são Membros do Inatel, mas cada categoria possui responsabilidades e formas de se identificar diferentes dentro do campus.

Tarefa:

1. Crie uma classe base chamada MembroInatel:
   - Adicione um atributo nomeCompleto (string).
   - Crie um método (virtual) chamado identificar(). O método deve exibir: "Sou um membro da comunidade Inatel: [nomeCompleto]."
2. Crie as classes Coordenador e Pesquisador que herdem de MembroInatel:
   - Adicione o atributo departamento (string) na classe Coordenador.
   - Adicione o atributo laboratorio (string) na classe Pesquisador.

EXERCÍCIO 3

3. Sobrescreva o método identificar() em ambas as classes:
   - No Coordenador, ele deve exibir: "Meu nome é [nomeCompleto], sou o coordenador do departamento de [departamento] no Inatel."
   - No Pesquisador, ele deve exibir: "Meu nome é [nomeCompleto], e realizo pesquisas no laboratório [laboratorio] do Inatel."
4. Na main:
   - Crie ponteiros da classe base MembroInatel.
   - Aponte o primeiro ponteiro para um objeto do tipo Coordenador e o segundo para um objeto do tipo Pesquisador.
   - Atribua valores aos atributos e chame o método identificar() através dos ponteiros da classe base para demonstrar o polimorfismo.

EXERCÍCIO 4

Cenário: Durante o Conselho de Paz das Terras Ancestrais, representantes de diferentes linhagens se reúnem. Embora cada um pertença a uma cultura distinta como Anões, Orcs e Draconatos, todos são reconhecidos como Membros do Conselho. Seu objetivo é criar um sistema que gerencie esses representantes em um único registro e permita que cada um faça sua saudação tradicional.

Tarefa:

1. Crie uma classe base chamada MembroConselho:
   - Adicione o atributo nome (string).
   - Crie um método (virtual) chamado saudar(). Dentro dele, imprima: "Um membro do conselho faz uma reverência silenciosa."

EXERCÍCIO 4

1. Crie três classes filhas que herdem de MembroConselho:
   - Anao: Sobrescreva saudar() para exibir: "Pela força da montanha, eu sou [nome] e honro nosso aço!"
   - Orc: Sobrescreva saudar() para exibir: "Pelo sangue e glória, eu sou [nome] e trago a força de minha tribo!"
   - Draconato: Sobrescreva saudar() para exibir: "Pelo sopro dos ancestrais, eu sou [nome] e falo em nome dos dragões!"
2. Na main:
   - Crie uma única lista para armazenar objetos do tipo MembroConselho.
   - Crie uma instância de cada raça (Anao, Orc, Draconato) com nomes diferentes e adicione-as à sua lista.
   - Percorra a lista, Para cada elemento, chame o método saudar() para demonstrar o comportamento polimórfico.

FIM DA AULA!

zSh3rl0cK/**S01-monitoria**

1 Contributor | 0 Issues | 0 Stars | 0 Forks

zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.

GitHub